



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

[Prova Finale] Progetto di Reti Logiche

Autore: **Matteo Rossi**

Matricola: 982793

Codice Persona: 10798975

Professore: Gianluca Palermo

Academic Year: 2023-24

Sommario

Sommario	i
1 Introduzione	1
1.1 Analisi della specifica	1
2 Architettura	2
2.1 FSM State Controller	3
2.2 K Counter Module	5
2.3 Parallel Parallel Register	6
2.4 Multiplexer	6
2.5 Adder	6
3 Risultati Sperimentali	7
3.1 Sintesi	7
3.2 Simulazioni	8
4 Conclusioni	11

1 | Introduzione

1.1. Analisi della specifica

La specifica di progetto richiede di implementare un modulo hardware in linguaggio VHDL che gestisca la comunicazione con una memoria e soddisfi determinate specifiche.

Sinteticamente, il sistema elabora una sequenza di parole rappresentate da valori numerici nell'intervallo $[0, 255]$, identificando l'assenza o mancata trasmissione di tale dato con il valore numerico nullo. Le parole sono memorizzate a partire da un indirizzo di memoria specifico a intervalli di due byte.

Il componente progettato completa questa sequenza ripristinando i valori mancanti con l'ultimo valore valido precedentemente memorizzato. Inoltre, associa, nella cella di memoria successiva, un valore di credibilità decrescente, basato sulla recente ricezione di un valore utile, dove 31 indica che il dato letto è sicuramente attendibile. Fornire un valore inferiore a zero non ha ulteriore significato e quindi la credibilità in questo caso resta invariata.

Possibili applicazioni e similarità con algoritmi realmente esistenti. Nel campo della trasmissione o ricostruzione dei dati, così come nella correzione degli errori, esistono algoritmi che ricordano il comportamento richiesto dalla specifica di progetto. Tuttavia, il nostro approccio non utilizza alcuna sequenza di valori ridondanti o bit di parità, ma si distingue per l'uso del valore di credibilità, che rappresenta la recente ricezione del dato.

Questo approccio potrebbe essere utile in applicazioni in cui non è possibile ottenere la ritrasmissione della sequenza ed è necessario fornire un valore il più veritiero e temporalmente vicino possibile, come nelle telecomunicazioni o nei sistemi di controllo industriale.

2 | Architettura

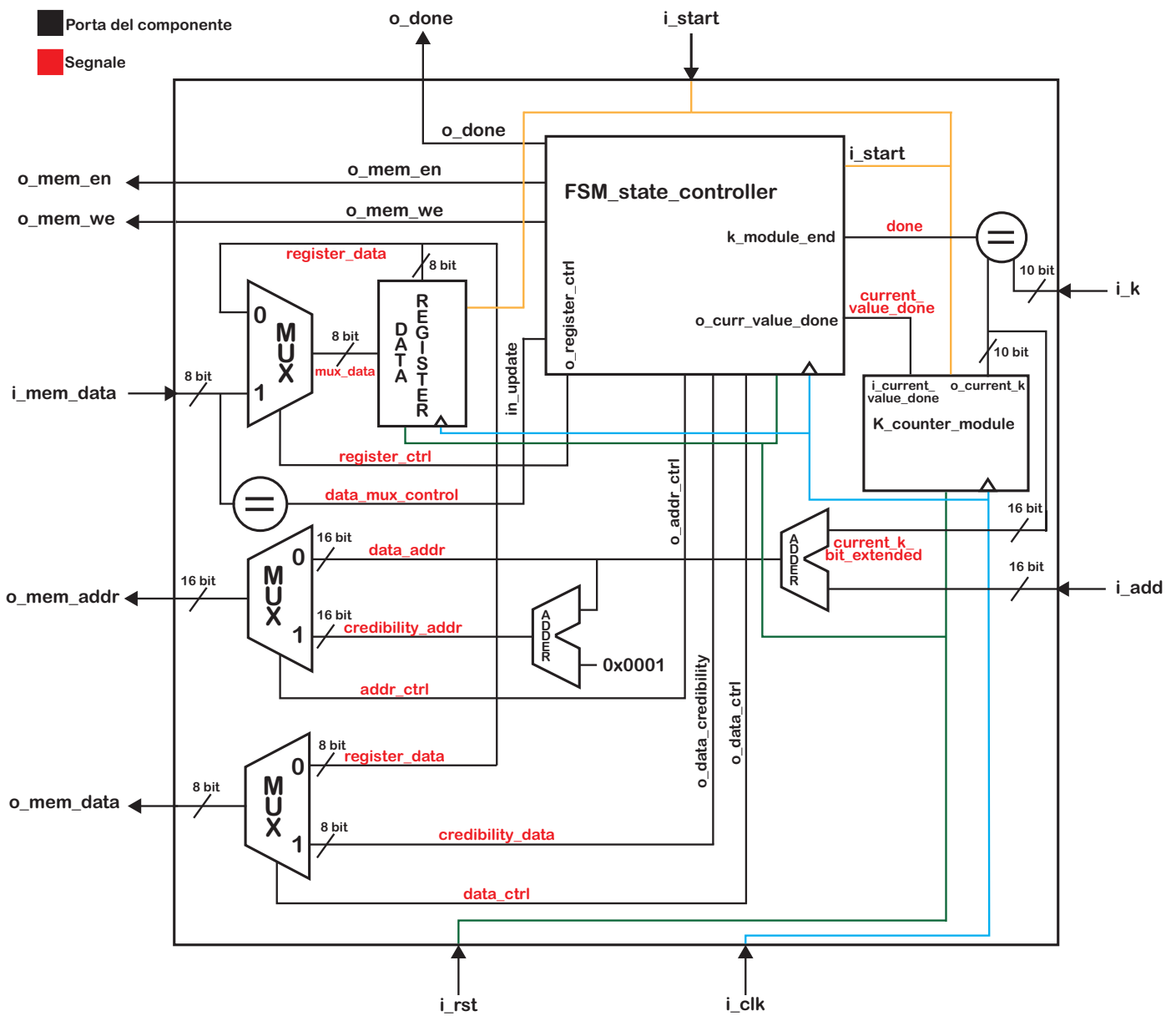


Figura 2.1: Rappresentazione concettuale dell'architettura del componente.

Descrizione dell'approccio progettuale. Il progetto è stato realizzato attraverso una progettazione modulare, suddividendo logicamente le operazioni necessarie all'implementazione della specifica in entità di dimensione minore, rispettivamente dedicate a uno dei singoli passi che compongono l'algoritmo.

2.1. FSM State Controller

La macchina a stati finiti è il modulo nevralgico dell'architettura, a cui è delegata l'intera generazione dei segnali relativi alla comunicazione con le porte del componente e i diversi moduli, così come la gestione dello stato corrente e la generazione del valore di credibilità esatto da associare alla parola in analisi. L'algoritmo si compone di 9 diversi stati:

- **IDLE-(RST)** - Il componente è in attesa di una variazione del segnale di start ed è pronto a svolgere l'elaborazione delle sequenze.
- **REQ_MEM_READ_VALUE** - Si richiede alla RAM la lettura del valore, il quale sarà poi disponibile al prossimo ciclo di clock. Viene anche verificato il caso di early termination in cui le word da elaborare siano nulle ($K=0$).
- **MEM_READ_VALUE** - Si analizza il dato fornito dalla memoria, verificando se si tratta di un valore non specificato o di una nuova parola. Questa valutazione è rappresentata attraverso il segnale *data_mux_control* e fornita alla FSM attraverso la porta *in_update*.
- **UPDATE_REGISTER_VALUE** - Attraverso il selettore del multiplexer, si aggiorna il valore nel data register con la nuova parola o si mantiene la precedente salvata.
- **UPDATE_CREDIBILITY_REGISTER** - Il registro che mantiene il valore di credibilità corrente viene riportato a 31 nel caso di nuova parola, decrementato o lasciato a 0 nel caso di valore non specificato.
- **MEM_WRITE_DATA** - Nel caso in cui il valore letto fosse nullo, si rende necessaria la scrittura del valore precedentemente letto nella sequenza in memoria al medesimo indirizzo.
- **MEM_WRITE_CREDIBILITY** - Scrittura del valore di credibilità all'indirizzo successivo. Si segnala al modulo contatore di k di incrementare il proprio registro.
- **UPDATE_K_REGISTER** - Aggiornamento del valore e valutazione della condizione di terminazione, che viene segnalata positivamente alla FSM nel caso in cui dovesse essere raggiunta.
- **DONE** - Attesa dell'abbassamento del segnale di start per la corretta comunicazione come stabilita da protocollo.

Concettualmente, il suo funzionamento è illustrato nella figura seguente:

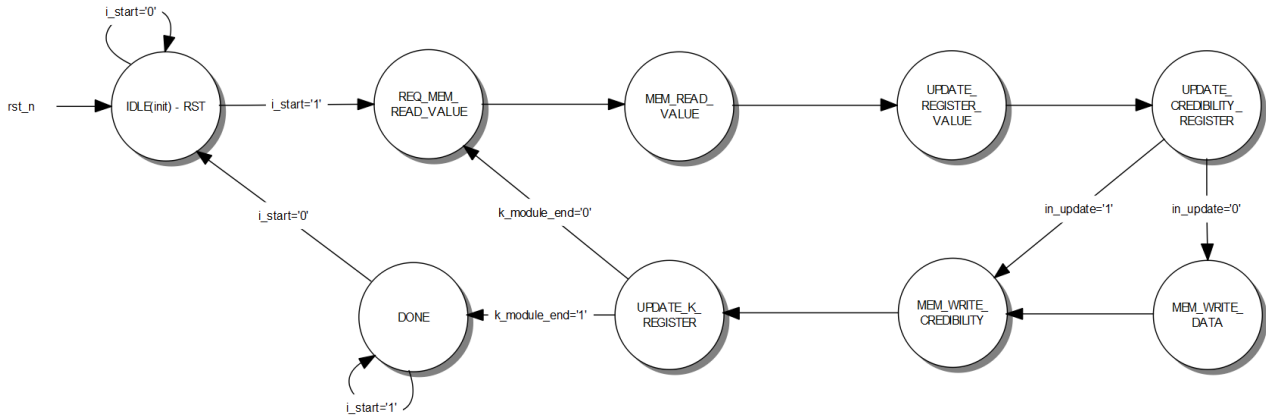


Figura 2.2: Diagramma degli stati della FSM.

La funzione δ , che determina lo stato prossimo della FSM, è realizzata attraverso un process, valutato a ogni variazione dei segnali di clock e reset.

Alias	Stato	δ : Stato Prossimo
S0	IDLE(init)-RST	if i_start = 1 then -> S1 else -> S0
S1	REQ_MEM_READ_VALUE	if k_module_end = 1 then -> S8 else -> S2
S2	MEM_READ_VALUE	-> S3
S3	UPDATE_REGISTER_VALUE	-> S4
S4	UPDATE_CREDIBILITY_REGISTER	if in_update = 1 then -> S6 else -> S5
S5	MEM_WRITE_DATA	-> S6
S6	MEM_WRITE_CREDIBILITY	-> S7
S7	UPDATE_K_REGISTER	if k_module_end = 1 then -> S8 else -> S2
S8	DONE	if i_start = 1 then -> S8 else -> S0

Tabella 2.1: Stato, Alias e Funzione di Stato Prossimo

Conseguentemente, anche la funzione λ , che aggiorna i segnali in uscita, è realizzata attraverso un process valutato a ogni variazione dello stato corrente.

	o_data_ ctrl	o_addr_ ctrl	o_register_ ctrl	o_curr_ value_done	o_mem_en	o_mem_we	o_done
S0	0	0	0	0	0	0	0
S1	0	0	0	0	1	0	0
S2	0	0	0	0	0	0	0
S3	0	in_update	0	0	0	0	0
S4	0	0	0	0	0	0	0
S5	0	0	0	0	1	1	0
S6	1	1	0	1	1	1	0
S7	0	0	0	0	0	0	0
S8	0	0	0	0	0	0	1

Tabella 2.2: Mappatura Stato - Segnale fornito in uscita

Nonostante sia possibile ridurre il numero di stati necessari e ottenere un comportamento che soddisfi ugualmente la specifica svolgendo parallelamente alcune delle operazioni, ho scelto di mantenere comunque un elevato numero di stati.

Questo è stato fatto principalmente per permettere una comprensione più chiara delle scelte implementative verso coloro che visionano il progetto, facilitare le modifiche progettuali dettate dalla necessità di apportare cambiamenti alla specifica e agevolare il lavoro da me compiuto durante la progettazione del componente, avendo sempre chiaro a quale stato io stessi apportando delle modifiche e quali altri moduli fossero coinvolti.

2.2. K Counter Module

L'entità, implementata attraverso un'architettura strutturale con istruzioni dataflow, si occupa di mantenere il numero di parole correntemente elaborate dal componente e fornisce in uscita i segnali per determinare la terminazione della valutazione di una sequenza, poiché è stato raggiunto il valore richiesto o il limite massimo di parole processabili (1024, dati i 10 bit del segnale i_k).

La comunicazione del modulo è unicamente verso la FSM e l'adder di indirizzamento della memoria.

L'incremento del valore viene effettuato al ciclo di clock in cui il segnale $o_current_value_done$ risulta attivo e la condizione di terminazione anticipata viene notificata prima di qualsiasi interazione di scrittura sulla memoria, lasciando inalterato lo scenario.

2.3. Parallel Parallel Register

Il salvataggio delle informazioni rilevanti al funzionamento dell'architettura, quali le parole lette dalla memoria, il valore di credibilità corrente e il numero di parole attualmente elaborate è stato realizzato mediante l'introduzione esplicita di tre registri (sincroni) paralleli a n bit, implementati mediante un'architettura behavioral. Il primo di questi è identificato come *data_register*, mentre i rimanenti sono incapsulati all'interno dei moduli FSM e *k_counter*, illustrati precedentemente.

All'entità tradizionale è stato inoltre aggiunta un'ulteriore porta per ricevere in ingresso il segnale di *i_start*, in modo da poter resettarne il contenuto per l'elaborazione di sequenze successive seguendo il segnale stesso, senza ricorrere a un reset esplicito.

2.4. Multiplexer

Vista la necessità di selezionare diversi indirizzi di memoria e dati da fornire alla RAM in base allo stato in cui il componente si trova e, inoltre, di gestire il flusso di dati in ingresso al data register, sono presenti 3 multiplexer a due operandi aventi numero di bit generico, il quale viene determinato dalla lunghezza del segnale trattato.

Il componente, implementato attraverso un'architettura dataflow, non presenta nessuna modifica aggiuntiva e riceve in ingresso un segnale di controllo per selezionare adeguatamente l'operando necessario.

2.5. Adder

Il calcolo degli indirizzi di memoria viene eseguito da 2 adder a due operandi aventi numero di bit fissato di default a 16, in quanto dimensione massima dei valori indirizzabili.

Il componente, anch'esso implementato attraverso un'architettura dataflow, non presenta nessuna modifica aggiuntiva e riceve in ingresso un indirizzo base, a cui viene sommato lo spiazamento.

Il primo adder, denominato *mem_addr_adder* riceve l'indirizzo base di memoria fornito da *i_addr* e il valore corrente di parole (word: K) computate dal componente, shiftato a sinistra di uno per ottenere la progressione corretta delle celle di memoria, ovvero ogni 2 byte per word ($ADD + 2 * (K - 1)$). Il secondo, *credibility_addr_adder*, effettua una banale operazione di incremento dell'ultimo bit, raffigurato attraverso il valore esadecimale 0x0001.

Gli output sono collegati al multiplexer dedicato alla gestione degli indirizzi, *mem_addr_mux*.

3 | Risultati Sperimentali

3.1. Sintesi

La tabella di report prende come riferimento la FPGA proposta nella specifica, anche se indicativa, vista la dimensione ridotta del componente sintetizzato.

Site Type	Used	Available	Util%
LUTs as Logic	82	134600	0.06
Register as FF	27	269200	0.01
Register as Latch	0	269200	0.00

Tabella 3.1: Report di utilizzo - Slice Logic, fornito da Vivado

E' possibile notare come il componente non presenti Latch, i quali potrebbero alterarne il corretto comportamento durante la fase di testing post-sintesi. Il numero di Flip-Flop è coerente con quanto ipotizzato nell'architettura teorizzata, infatti:

Modulo	# FF
K Counter Module	10
Data Register	8
Credibility Counter	5
Stati FSM	$\lfloor \log_2 9 \rfloor = 4$
	27

Tabella 3.2: Conteggio dei Flip-Flop

Il Timing Report indica che il componente ha uno *Slack Time* (Required Time - Arrival Time) di 16.650 ns a fronte di un segnale di clock di 20 ns, rendendo il componente adatto a lavorare anche con limitazioni temporali molto più stringenti. Questo tradeoff è possibile a fronte di una maggiore richiesta di cicli di clock complessivi per compiere l'elaborazione, dettati dalla necessità di scorrere gli stati. Il data path delay medio è di 3.199 ns (di cui 1.425 ns di Logic e 1.774 ns di route, equamente distribuito) e la sintesi presenta 3 livelli di logica.

3.2. Simulazioni

Commento ai Testbench proposti Preliminarmente, la fase di verifica del funzionamento del componente sintetizzato ha visto la copertura delle casistiche previste dal testbench proposto e dai casi d'esempio illustrati nella specifica. In particolare, le condizioni logicamente verificate sono costituite da:

- **Esempio 0,1** Normale controllo dell'elaborazione di una sequenza di valori generica.
- **Esempio 2** Verifica della copertura di tutti i valori di credibilità attraverso una sequenza di lunghezza superiore all'intervallo di rappresentazione ($K = 35$), con conseguente scrittura del valore C:0 nelle ultime parole.
- **Esempio 3** Inizio della sequenza caratterizzato da un valore nullo, il quale implica l'impossibilità di salvare un valore numerico iniziale nel registro e la scrittura del valore C:0. Il test copre anche i valori estremi possibili di $W \in [0,255]$.
- **Esempio 4** Sequenza di elaborazione che presenta la combinazione di alcune casistiche precedentemente discusse.
- **Esempio 5** Sequenza di parole non nulle in memoria, tutte diverse dalle precedenti.

Inoltre, l'insieme verifica diversi valori di K e ADD.

Analisi di ulteriori casistiche L'ideazione di ulteriori casi di test è stata guidata principalmente dalla verifica del rispetto del protocollo di comunicazione e da condizioni al limite sui segnali in ingresso forniti al componente e generati o salvati durante l'esecuzione.

Entrando più nel dettaglio:

- **TB ALL ZEROS**
La memoria contiene K:32 word nulle. L'output previsto è la scrittura rispettivamente di 32 valori di credibilità nulli.
- **TB NO ZEROS IN MEMORY-ALL DIFFERENT WORDS**
Come illustrato nell'Esempio 5, ulteriori sequenze che rispettano il medesimo criterio.
- **TB NO ZEROS IN MEMORY-ALL SAME WORD**
Lieve modifica concettuale dove la parola scritta in memoria è sempre la stessa. L'output previsto è la scrittura di valori aventi credibilità C:31.
- **TB FIRST WORD THEN ALL ZEROS**
Come nell'Esempio 2, modificando arbitrariamente la lunghezza delle sequenze e l'indirizzo base di memoria.
- **TB K=0**
Condizione al limite in cui si accerta la capacità del componente di terminare precedentemente l'elaborazione data una sequenza di lunghezza nulla e di lasciare immutato lo scenario caricato in memoria. Potrebbe simulare una mancata trasmissione del segnale.
- **TB K=1023**
Massimo valore di lunghezza della sequenza processabile dal componente. Verifica la

capacità del *k_counter_module* di rappresentare il valore limite e di fornire poi il segnale di terminazione.

- **TB W = 255**

Massimo valore in lettura dalla memoria. Verifica la capacità del registro dati di salvare il valore limite. Il test è ridondante rispetto a più casi d'esempio, che già comprendono il medesimo valore.

- **TB ELABORAZIONI CONSECUTIVE**

Verifica il rispetto del protocollo di comunicazione e il reset dei moduli tra più elaborazioni. Si suppone che l'innalzamento del segnale *i_start* avvenga almeno dopo un ciclo di clock, per permettere al componente di abbassare *i_done*. Inoltre, l'i-esima elaborazione inizia con alcune word nulle in memoria, per verificare che non resti traccia nei registri di valori precedenti, nonostante l'assenza di un segnale di RST esplicito.

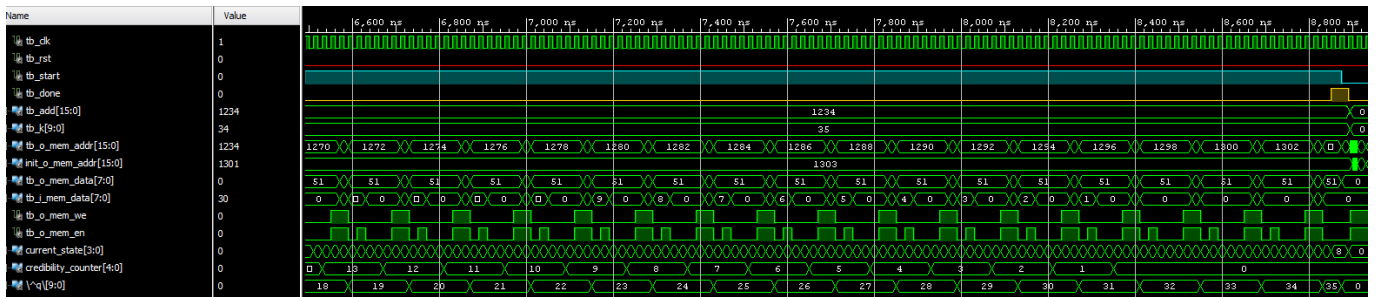


Figura 3.1: Post-Synthesis Functional Simulation - Analisi dei segnali durante l'elaborazione.

E' possibile notare come il componente costruisca gli indirizzi di memoria sommando all'indirizzo base lo spiazzamento salvato dal *k_counter_module*, richieda la lettura in un ciclo di clock e ne impieghi due per riscrivere eventualmente il valore corretto e successivamente la credibilità. Inoltre, una volta che il segnale di *i_start* si abbassa, tutti i segnali rilevanti vengono resettati e lo stato corrente della FSM diventa IDLE-(RST).

- **TB ULTIMO INDIRIZZO DI MEMORIA DISPONIBILE**

Verifica della corretta gestione dell'ultimo valore di memoria indirizzabile dal componente: 0xFFFF. L'obiettivo è verificare la copertura di tutto il range di indirizzi utilizzabile.

- **TB RST ASINCRONO A T CASUALE E NUOVA ELABORAZIONE**

Il componente viene stimolato in risposta a un segnale di RST fornito in diverse modalità, poiché non è chiaro il comportamento assunto dal segnale *i_start* in questo contesto. L'obiettivo è valutare la correttezza delle elaborazioni, anche eseguendo scritture successive sulla medesima area di memoria.

1. RST - Start High. Il segnale di start resta alto e viene elaborato un nuovo scenario. Il componente esegue l'elaborazione in una nuova sezione di memoria.
2. RST - Start Low. Il segnale di start si abbassa e il testbench effettua il caricamento di un nuovo scenario. Lo stato della memoria viene valutato al termine della elaborazione.

4 | Conclusioni

Il componente hardware implementato è conforme alle specifiche e ai vincoli imposti sul segnale di clock, garantendo la correttezza delle elaborazioni. Inoltre, non si evidenziano criticità e tutte le simulazioni dei casi limite precedentemente previsti si sono dimostrate accurate durante la fase di sintesi.

In aggiunta, sebbene non fosse un requisito di progetto, il componente è stato implementato e tutti i testbench sono stati simulati correttamente sia in Post-Implementation Functional che Timing.

Si ritiene pertanto di aver realizzato un componente in grado di gestire le possibili richieste descrivibili in linea con la specifica fornita. Il componente è in grado di adattarsi a contesti in cui possono verificarsi segnali di reset in vari punti dell'elaborazione, garantendo così una maggiore affidabilità. La modularizzazione delle principali azioni che compongono l'algoritmo ha contribuito a migliorare la leggibilità e la manutenibilità del codice.